Guía para Diagramas de Secuencia

https://gemini.google.com/app/f3a5be4b54cb0a42

User prompt: Quiero hacer mi diagrama de secuenceia de mi proyecto, para que me guies o si puede dame un modelo, que ya lo estoy creando en lucichart?

Response: Para ayudarte con tu **Diagrama de Secuencia** en Lucidchart, lo ideal es estructurarlo en tres partes clave: la **arquitectura en capas (Lifelines)**, el **flujo estándar** de comunicación, y los **modelos prácticos** que puedes copiar directamente a tu lienzo.

1. Estructura recomendada para las Líneas de Vida (*Lifelines*)

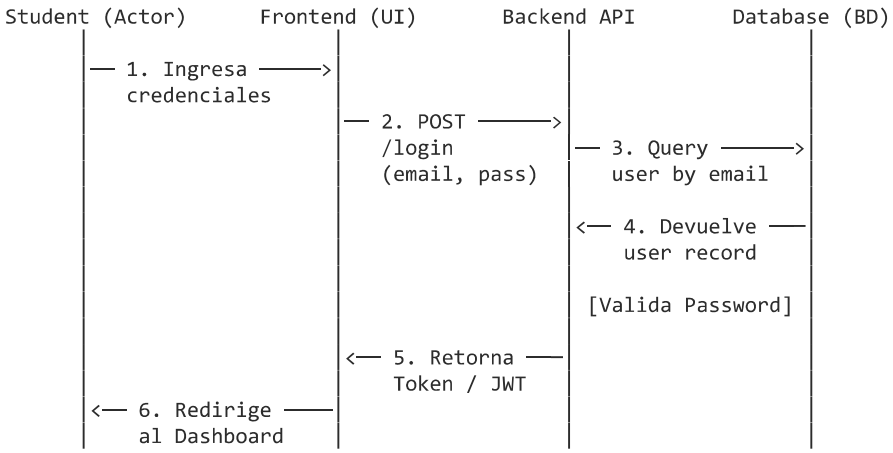
En tu diagrama de Lucidchart, debes colocar de izquierda a derecha los siguientes componentes para representar una arquitectura *Clean/MVC* estándar:

1. **`Actor`**: Usuario que desencadena la acción (ej. `Student`, `Company` o `Admin`).
2. **`Frontend / UI`**: La interfaz de usuario (ej. React, Angular o Flutter).
3. **`Backend API`**: El controlador central o servidor API (ej. Node.js, Express, FastAPI).
4. **`Auth / Engine Service`**: Servicios específicos del backend (ej. servicio de autenticación o motor de recomendación).
5. **`Database (BD)`**: La base de datos del sistema (donde consultamos las tablas como `STUDENT`, `JOB`, etc.).

2. Modelos Prácticos para Lucidchart

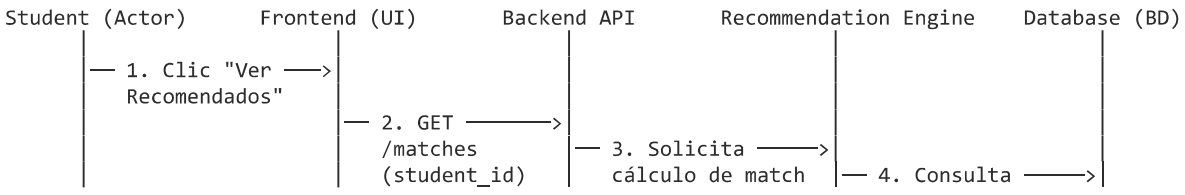
A continuación tienes dos modelos paso a paso para los flujos principales de tu plataforma:

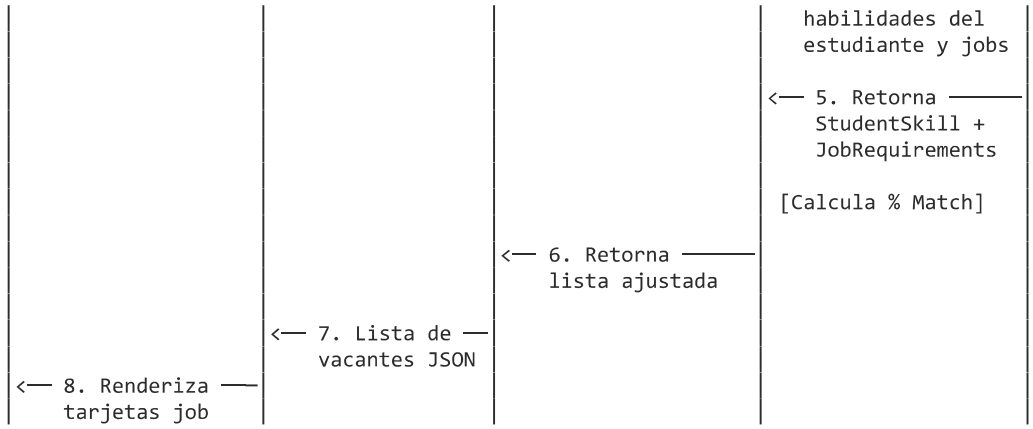
Modelo A: Flujo de Inicio de Sesión (*Login*)



- **Mensaje 1 (Sólido →):** `Ingresar credenciales (email, password)`
- **Mensaje 2 (Sólido →):** `POST /api/auth/login`
- **Mensaje 3 (Sólido →):** `SELECT * FROM STUDENT WHERE email = ?`
- **Mensaje 4 (Punteado ⇠):** `Datos de usuario y Hash de contraseña`
- **Operación interna en Backend:** *Verificar contraseña Hash y generar JWT Token*.
- **Mensaje 5 (Punteado ⇠):** `Respuesta 200 OK + JWT Token`
- **Mensaje 6 (Punteado ⇠):** `Muestra Dashboard del Estudiante`

Modelo B: Flujo de Motor de Matching / Recomendación de Empleos





3. Buenas Prácticas al Dibujarlo en Lucidchart

1. **Uso de Líneas Continuas vs. Punteadas:**
   - Usa **líneas continuas con flecha rellena (⟶)** para llamadas síncronas o peticiones de un componente a otro (ej. peticiones HTTP o *queries*).
   - Usa **líneas punteadas con flecha abierta (⇠)** únicamente para las respuestas de datos o confirmaciones (*return messages*).
2. **Barras de Activación:**
   - Agrega el rectángulo delgado en la línea de vida solo durante el tiempo exacto que la entidad está ejecutando una tarea o procesando información.
3. **Manejo de Excepciones (*Fragmento ALT*):**
   - En Lucidchart, puedes arrastrar una caja de contenedor tipo `alt` (Alternativa / If-Else) si deseas mostrar qué ocurre cuando el usuario falla el inicio de sesión o no se encuentran empleos.

¿Qué caso de uso en particular estás diagramando en este momento en Lucidchart, o te gustaría que adaptemos algún otro flujo específico de tu sistema?

User prompt: ESA GRAFICA TIENE RELACION CON LA UML ?

Response: **Sí, totalmente.** Ese gráfico (el Diagrama de Secuencia) es **uno de los 14 diagramas oficiales que forman parte de UML** (*Unified Modeling Language* o Lenguaje Unificado de Modelado).

Para entender su lugar exacto dentro del estándar UML, ten en cuenta cómo se clasifica:

1. Su lugar dentro del Estándar UML

En la especificación **UML 2.5**, los diagramas se dividen en dos grandes familias:

1. **Diagramas Estructurales** (*Vista estática*: cómo está construido el sistema, como el *Diagrama de Clases* o *Diagrama de Componentes*).
2. **Diagramas de Comportamiento** (*Vista dinámica*: cómo reacciona o cambia el sistema con el tiempo).
   - Dentro de los diagramas de comportamiento, existe una subcategoría llamada **Diagramas de Interacción**.
   - **El Diagrama de Secuencia pertenece a los Diagramas de Interacción.**

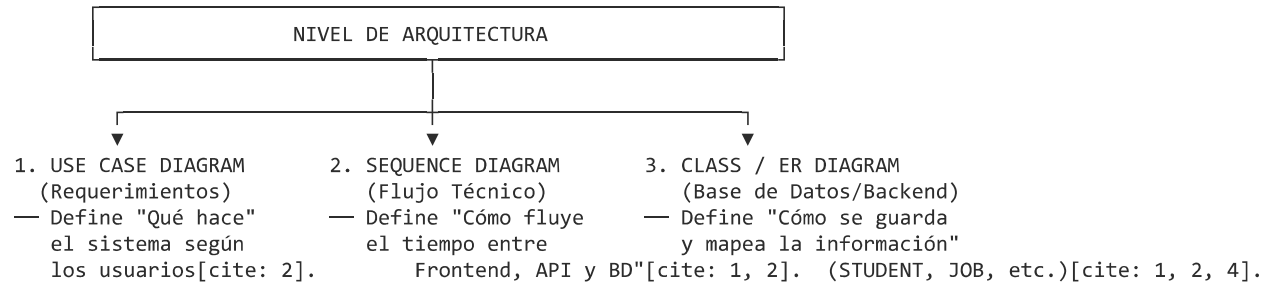
2. ¿Qué aporta el Diagrama de Secuencia dentro de UML?

A diferencia de otros diagramas de UML:

- **Diagrama de Casos de Uso:** Muestra *qué* hace el sistema desde el punto de vista del usuario (los actores y los óvalos).
- **Diagrama de Clases / E/R:** Muestra *qué datos o estructuras* existen (las tablas `STUDENT`, `JOB`, etc.).
- **Diagrama de Secuencia:** Muestra **el orden cronológico exacto paso a paso** de cómo interactúan las capas del software (Frontend, Backend API, Base de Datos) para cumplir un caso de uso específico.

3. Los 3 Diagramas UML clave para tu rol de *Software Engineer*

Aunque UML tiene 14 diagramas en total, en tu *Senior Project* tu responsabilidad como *Software Engineer* se concentra en estos tres:



Por lo tanto, el trabajo que estás haciendo en Lucidchart para los flujos de *Login* y *Matching* cumple directamente con el estándar formal de UML para diseño de software.

User prompt: pero que tal algo como esto

Response: El diagrama de la imagen tiene **varios errores técnicos graves de UML y fallas de flujo/lógica** que impiden que sea funcional y profesional. Para que te sirva en tu proyecto de ingeniería, **necesita correcciones profundas de estructura, mensajes y participantes**.

A continuación te detallo los errores específicos detectados y cómo debes reestructurarlo:

1. Errores Técnicos y Lógicos detectados en la imagen

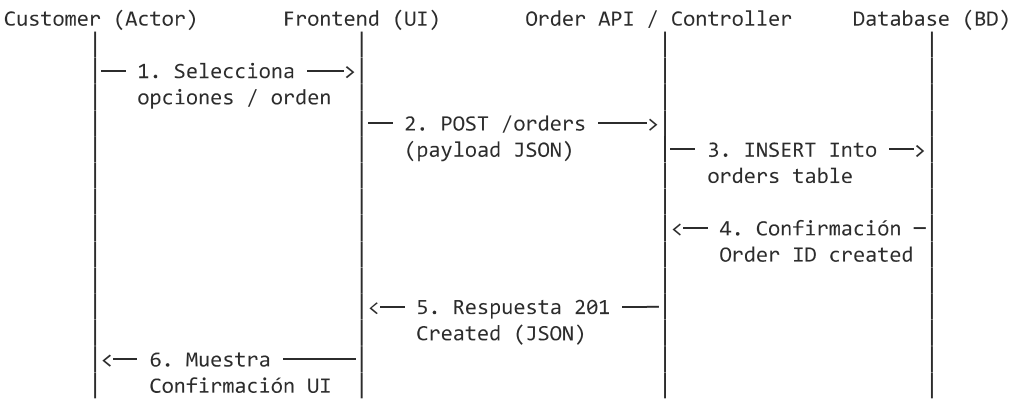
- Líneas de Vida (*Lifelines*) desordenadas y mal ubicadas:
  - Tienes a un actor `Employee` en **medio del diagrama** intercalado entre componentes de software. Los actores humanos siempre deben ir en los extremos (usualmente a la izquierda).
  - Tienes rectángulos genéricos con el texto `"Text"` que no representan ninguna entidad ni servicio del sistema.
- Flujos de mensajes incoherentes e invertidos:
  - El mensaje `Select Toppings` sale flotando desde el espacio entre `Employee` y `Login Service` directamente hacia `Order Service` cruzando todo el diagrama.
  - Hay una respuesta `Order Confirmed` apuntando desde `Order Service` hacia `Login Service`. Un servicio de login no procesa pedidos de comida.
  - La línea `Insert User` atraviesa por detrás del actor `Employee` hasta llegar a `Login Service`.
- Falta de activación adecuada:
  - Salen mensajes y respuestas de `Login Service` y `Online Delivery System` en zonas donde la barra de activación vertical ni siquiera ha empezado.

2. Estructura Profesional Correcta (Aplicada a tu Plataforma)

Un Diagrama de Secuencia profesional para un flujo de pedido/servicio o autenticación debe ordenarse **estrictamente de izquierda a derecha por capas**:

$$\text{Actor (Usuario)} \longrightarrow \text{Frontend (UI)} \longrightarrow \text{Backend API} \longrightarrow \text{Servicios / BD}$$

Ejemplo: Flujo Correcto de Procesamiento de Orden / Servicio



3. Recomendaciones inmediatas para tu diseño en Lucidchart

1. **Elimina las cajas con el texto** `"Text"` y reemplázalas por entidades reales de software (ej. `Frontend UI`, `Order API`, `Database`).
2. **Reorganiza las columnas:** Todos los usuarios/actores a la izquierda, la interfaz al lado, la lógica de servidor en el centro y las bases de datos a la derecha.
3. **No cruces flechas sobre otros componentes:** Si el cliente realiza una acción en la pantalla, la flecha va **únicamente** del `Actor` al `Frontend`.
4. **Respeta los tipos de flecha:**
    - Flecha sólida con punta rellena (⟶) para enviar peticiones.
    - Flecha punteada (⇠) para devolver datos/respuestas.

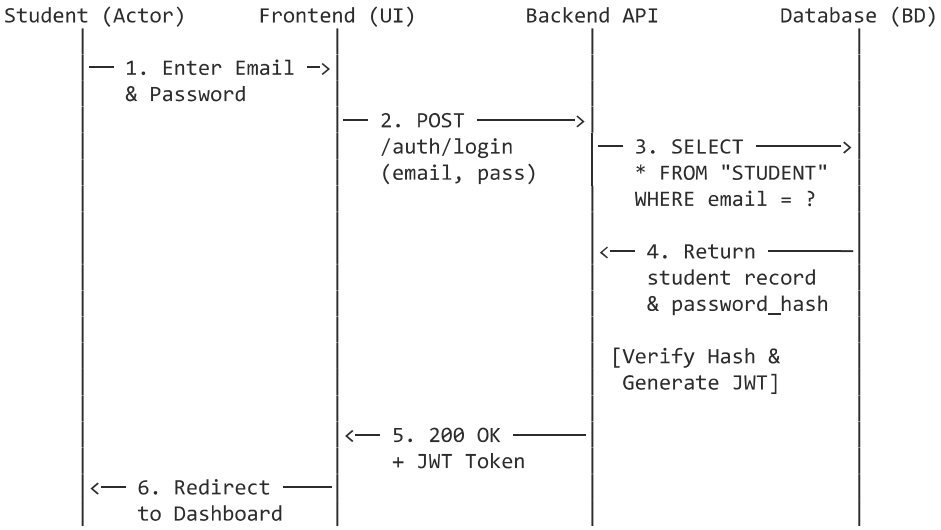
¿Quieres que rehagamos este diagrama enfocado en el flujo de pedidos/ordenes o prefieres adaptarlo directamente a un caso de uso específico de tu plataforma (como Login o Recomendaciones)?

User prompt: la grafica que me habias dado primero pero damelo en ingles

Response: Here are the two sequence diagram models in **English**, updated to align with the specific architecture, endpoints, database entities, and matching logic of your **Compass** project:

1. Model A: Student Authentication Flow (Login)

This diagram shows the sequence when a user logs in to receive a JWT token from the backend.

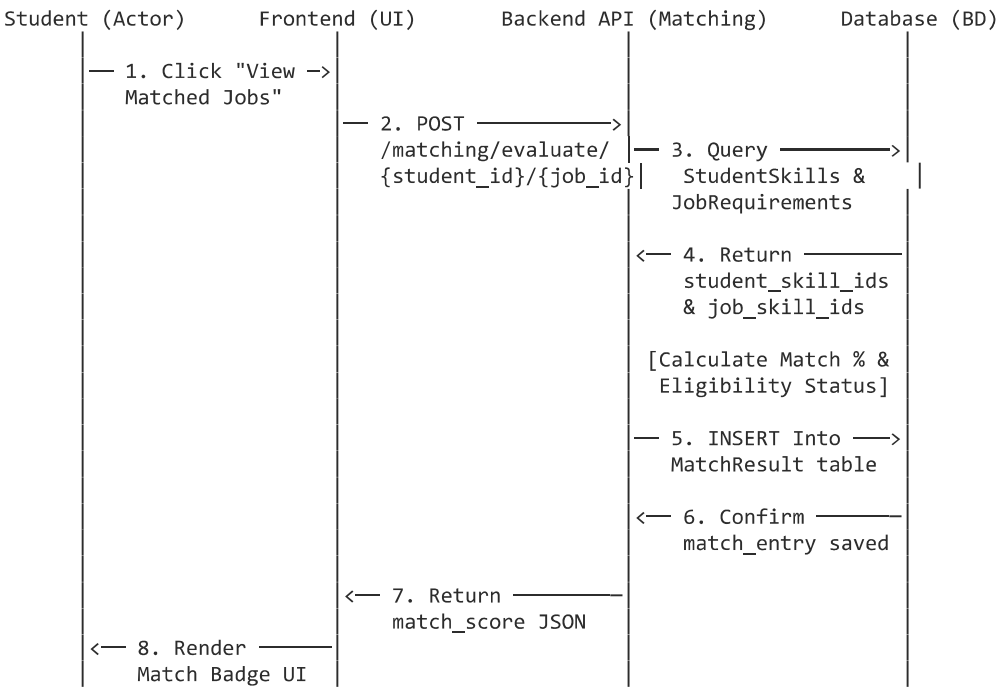


Detailed Message Steps for Lucidchart:

- **Message 1 (Solid Line ⟶):** `Enter credentials (email, password)`
- **Message 2 (Solid Line ⟶):** `POST /auth/login`
- **Message 3 (Solid Line ⟶):** `SELECT student_id, email, password_hash FROM "STUDENT" WHERE email = ?`
- **Message 4 (Dashed Line ⇠):** `Student record (password_hash)`
- **Self-Message / Internal Execution Box:** `Verify password hash & generate JWT Token`
- **Message 5 (Dashed Line ⇠):** `200 OK (access_token, token_type)`
- **Message 6 (Dashed Line ⇠):** `Render Student Dashboard`

2. Model B: Matching Engine & Evaluation Flow (`routers/matching.py`)

This diagram models the automated compatibility evaluation between a student's skills and a job vacancy's requirements.



Detailed Message Steps for Lucidchart:

- **Message 1 (Solid Line →):** `Click "Check Job Matches"`
- **Message 2 (Solid Line →):** `POST /matching/evaluate/{student_id}/{job_id}`
- **Message 3 (Solid Line →):** `SELECT skill_id FROM StudentSkill WHERE student_id = ?` & `SELECT skill_id FROM JobRequirement WHERE job_id = ?`
- **Message 4 (Dashed Line ←--):** `List of student_skill_ids and job_skill_ids`
- **Self-Message / Internal Execution Box:** `Calculate match_score % & determine eligibility_status ("Eligible", "Needs_Training", "Not_Eligible")`
- **Message 5 (Solid Line →):** `INSERT Into MatchResult (student_id, job_id, match_score, eligibility_status)`
- **Message 6 (Dashed Line ←--):** `MatchResult record created (match_id)`
- **Message 7 (Dashed Line ←--):** `200 OK Response (match_score: 85.0%, eligibility_status: "Eligible")`
- **Message 8 (Dashed Line ←--):** `Display Compatibility Score Card`

Best Practices for Lucidchart Lifelines & Shapes

Layer Role	UML Shape Type in Lucidchart	Lifeline Title
Actor	Actor Symbol (⊙ —|)	`Student` / `Company` / `Admin`
User Interface	Boundary Class (◯⊢) or Standard Lifeline	`Frontend Application (UI)`
API Controller	Control Class (◯⟲) or Standard Lifeline	`Backend API (FastAPI)`
Database	Entity Class / Database Cylinder (θ)	`Database (PostgreSQL)`

User prompt: Por que me salen dos Models?

Response: Te salen dos modelos en el ejemplo porque representan **dos casos de uso o flujos de trabajo (*use cases*) completamente distintos** dentro de tu proyecto **Compass**:

1. Modelo A: Autenticación (*Login*)

- **Propósito:** Mostrar el flujo paso a paso cuando un usuario (`Student`, `Company` o `Admin`) inicia sesión en la plataforma.
- **Participantes involucrados:** Solo el Usuario, el Frontend, el Backend API y la tabla `STUDENT` de la Base de Datos.
- **No requiere el motor de matching**, ya que en este punto únicamente se están validando las credenciales (email y contraseña hash) para entregar el token de acceso JWT.

2. Modelo B: Evaluación de Compatibilidad (*Matching Engine*)

- **Propósito:** Mostrar la lógica técnica de negocio cuando se evalúa qué tan compatible es un estudiante con una oferta laboral vacante (`JOB`).
- **Participantes involucrados:** Incluye un componente adicional de software en el backend: el **`Recommendation / Matching Engine`** (`routers/matching.py`).
- **Lógica del flujo:**
  1. Consulta las habilidades registradas del estudiante (`StudentSkill`).
  2. Consulta los requisitos de la vacante (`JobRequirement`).
  3. Ejecuta el algoritmo para calcular el porcentaje de compatibilidad (`match_score`).
  4. Guarda el resultado en la tabla `MatchResult` en la base de datos.

¿Cuál de los dos debes elegir para tu Lucidchart?

- Si tu profesor o entregable te pide **representar el acceso/autenticación al sistema**, debes dibujar el **Modelo A**.
- Si te piden **el núcleo / valor principal de tu sistema** (cómo funciona la recomendación y emparejamiento inteligente de tu proyecto **Compass**), debes dibujar el **Modelo B**.
- Si estás armando el documento de arquitectura completo, **lo ideal es incluir ambos diagramas por separado**, etiquetando cada uno con el título del caso de uso correspondiente (ej. *Sequence Diagram: Authentication* y *Sequence Diagram: Job Matching Evaluation*).

User prompt: yo cambien el UML, TE VOY a dar las graficas para que lo puedas analizar y hacer uno completo

Response: He analizado en detalle los diagramas de **Casos de Uso (UML Use Case Diagram)** que adjuntaste.

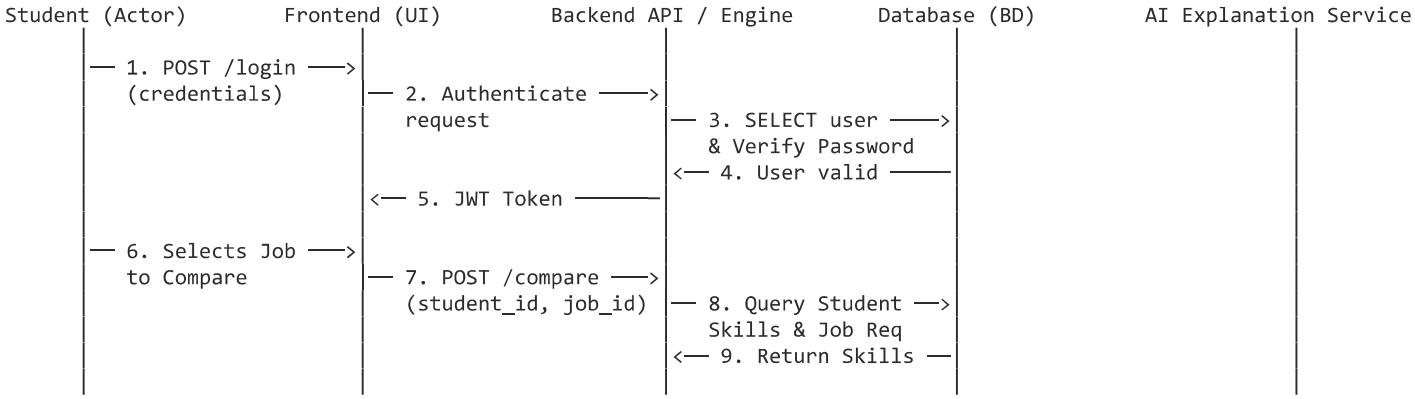
Tus diagramas dividen perfectamente el sistema en tres actores con responsabilidades bien delimitadas:

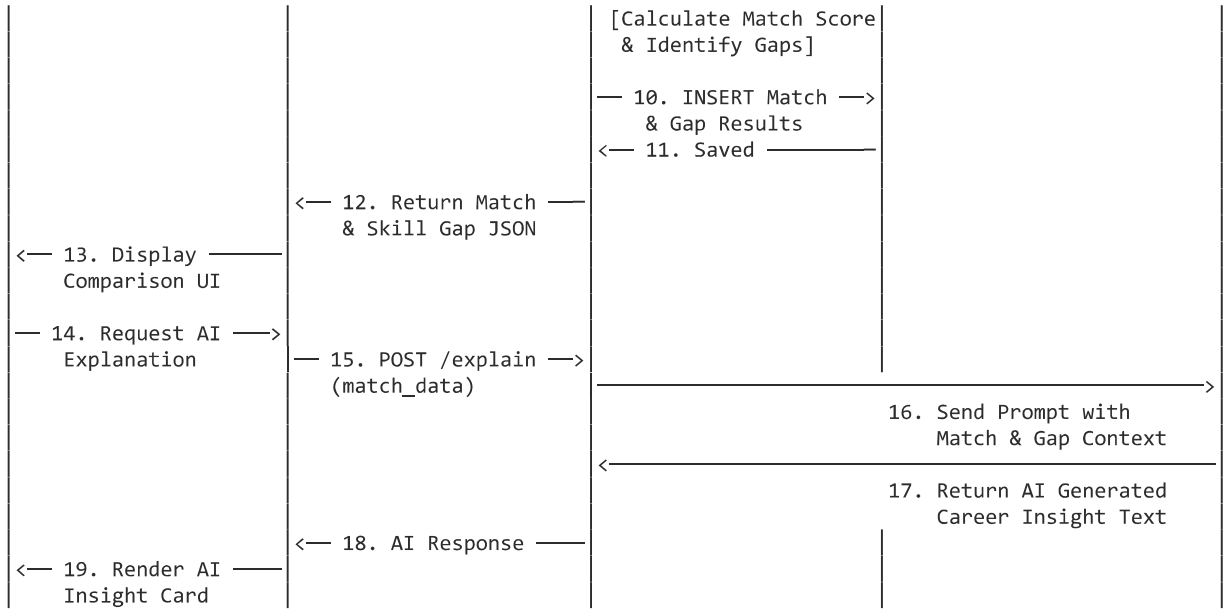
1. **`Students` (Estudiante):** Gestión de perfil, habilidades, exploración, comparación de *skills*, cálculo de *match score*, identificación de brechas (*skill gaps*), visión de cursos recomendados y consulta de explicación con IA.
2. **`COMPASS ADMINISTRATOR` (Administrador):** Administración de cuentas (`Manage Accounts`), gestión de roles (`Manage Roles`) e inicio de sesión (`Login` con verificación de contraseña y manejo de errores).
3. **`Content Maintainer` (Mantenedor de Contenido):** Gestión de perfiles de carrera (`manage career profiles`), *insights* profesionales, recursos de aprendizaje, revisión y publicación de contenido, y ofertas laborales (`manage job listings`).

Diagrama de Secuencia Completo e Integrado (*Sequence Diagram*)

Basado en la estructura de tu Diagrama de Casos de Uso, a continuación tienes el **Diagrama de Secuencia técnico completo** escrito en **inglés** (listo para copiar y estructurar en Lucidchart).

Este diagrama cubre el **flujo principal e integral del Estudiante** (autenticación, comparación de habilidades, cálculo de *match score*, brechas de conocimiento y recomendación de cursos/explicación IA):





Desglose de Mensajes para Lucidchart paso a paso

Fase 1: Autenticación (`Login` & `Verify Password`)

1. **`Student` → `Frontend` (Solid Line):** `Enter credentials (email, password)`
2. **`Frontend` → `Backend API` (Solid Line):** `POST /api/auth/login`
3. **`Backend API` → `Database` (Solid Line):** `SELECT * FROM "USER" WHERE email = ?`
4. **`Database` ←-- `Backend API` (Dashed Line):** `Return user record & password hash`
5. **`Backend API` ←-- `Frontend` (Dashed Line):** `200 OK + JWT Access Token`

Fase 2: Comparación de Habilidades y Motor de Coincidencia (`Compare Skills`, `Calculate Match Score`, `Identify Skill Gaps`)

6. **`Student` → `Frontend` (Solid Line):** `Click "Compare Skills to Job Listing"`
7. **`Frontend` → `Backend API` (Solid Line):** `POST /api/matching/compare (student_id, job_id)`
8. **`Backend API` → `Database` (Solid Line):** `SELECT StudentSkills & JobRequirements`
9. **`Database` ←-- `Backend API` (Dashed Line):** `Return skill matrices`
10. **`Backend API` Internal Execution Box:** `Calculate match_score % & extract missing skills (skill_gaps)`
11. **`Backend API` → `Database` (Solid Line):** `INSERT INTO MatchResult (match_score, skill_gaps)`
12. **`Database` ←-- `Backend API` (Dashed Line):** `Record saved confirmation`
13. **`Backend API` ←-- `Frontend` (Dashed Line):** `Return JSON (match_score, gaps, recommended_courses)`
14. **`Frontend` ←-- `Student` (Dashed Line):** `Render Comparison View & Gap Analysis`

Fase 3: Explicación y Orientación con IA (`Student requests AI explanation`)

15. **`Student` → `Frontend` (Solid Line):** `Click "Ask AI to Explain Gap Results"`
16. **`Frontend` → `Backend API` (Solid Line):** `POST /api/ai/explain-gap (student_id, match_id)`
17. **`Backend API` → `AI Explanation Service` (Solid Line):** `Send prompt payload with match_score & skill_gaps context`
18. **`AI Explanation Service` ←-- `Backend API` (Dashed Line):** `Return generated career advice & insight explanation`
19. **`Backend API` ←-- `Frontend` (Dashed Line):** `200 OK Response (ai_insight_text)`

20. **`Frontend` ←-- `Student` (Dashed Line):** `Display AI Career Insights Modal`

Configuración de Objetos / Líneas de Vida (*Lifelines*) en Lucidchart

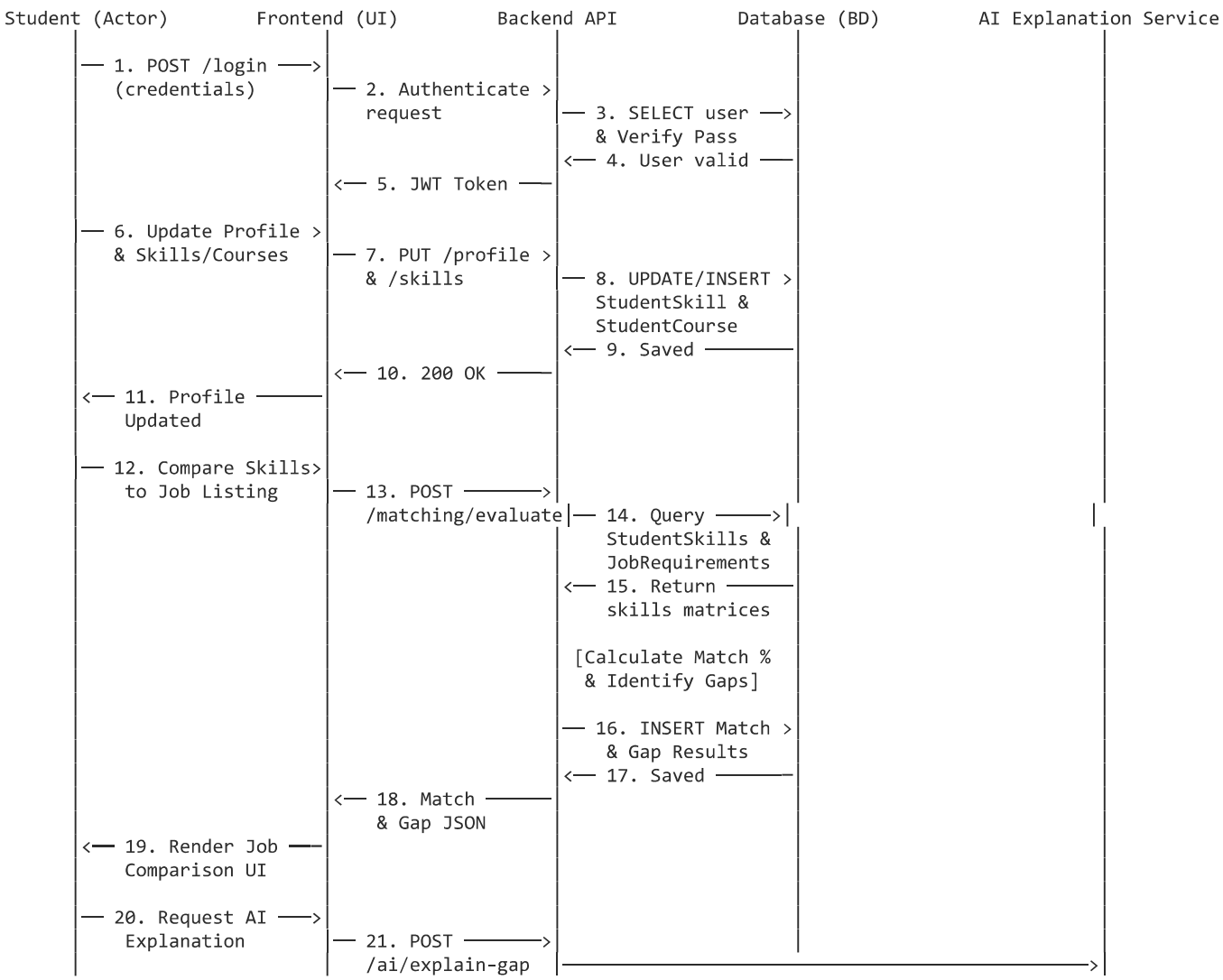
Componente UML	Tipo de Forma en Lucidchart	Nombre exacto de la columna
Actor	Actor Symbol (☉ —|)	`Student`
UI Layer	Boundary / Standard Lifeline	`Frontend (React / UI)`
Controller / Logic	Control Class / Standard Lifeline	`Backend API & Matching Engine`
Database Layer	Database Cylinder / Entity Class	`Database (PostgreSQL)`
External Service	Standard Lifeline / External Box	`AI Explanation Service (LLM API)`

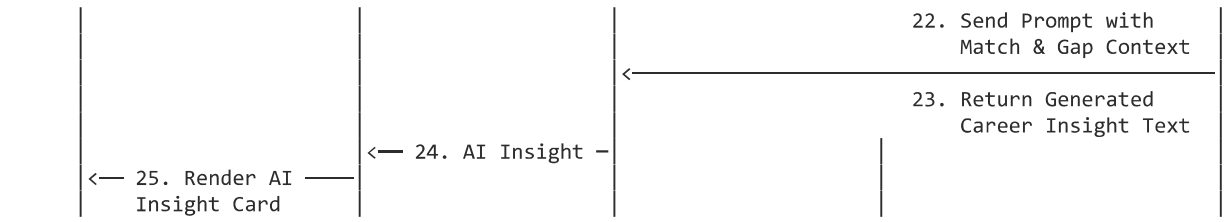
User prompt: pero no deberia tener la fase 2 para agregar el perfil o actulizarlo

Response: Tienes toda la razón. De acuerdo a tus casos de uso (`Manage Own Profile` y `Update Skills & Courses`), el flujo real del usuario **requiere registrar/actualizar su perfil y habilidades** antes o después de autenticarse, para que el motor de emparejamiento (*Matching Engine*) tenga datos que procesar.

Para que tu Diagrama de Secuencia en **Lucidchart** esté completo y refleje exactamente la secuencia del **Caso de Uso del Estudiante**, debe ordenarse cronológicamente en las siguientes fases:

Diagrama de Secuencia Completo (con Fase de Perfil y Habilidades)





Desglose de Pasos para Lucidchart (en Inglés)

Phase 1: Authentication (`Login` & `Verify Password`)

1. **`Student`** → **`Frontend`** **(Solid Line):** `Enter credentials (email, password)`
2. **`Frontend`** → **`Backend API`** **(Solid Line):** `POST /auth/login`
3. **`Backend API`** → **`Database`** **(Solid Line):** `SELECT * FROM "STUDENT" WHERE email = ?`
4. **`Database`** ←-- **`Backend API`** **(Dashed Line):** `Return user record & password_hash`
5. **`Backend API`** ←-- **`Frontend`** **(Dashed Line):** `200 OK + JWT Access Token`

Phase 2: Profile & Skills Management (`Manage Own Profile` & `Update Skills & Courses`)

6. **`Student`** → **`Frontend`** **(Solid Line):** `Update academic major, completed courses, & technical skills`
7. **`Frontend`** → **`Backend API`** **(Solid Line):** `PUT /students/me/profile` & `POST /students/me/skills`
8. **`Backend API`** → **`Database`** **(Solid Line):** `UPDATE "STUDENT" & INSERT INTO "StudentSkill" / "StudentCourse"`
9. **`Database`** ←-- **`Backend API`** **(Dashed Line):** `Confirm records updated`
10. **`Backend API`** ←-- **`Frontend`** **(Dashed Line):** `200 OK (Profile & Skills Updated)`
11. **`Frontend`** ←-- **`Student`** **(Dashed Line):** `Render Updated Profile Dashboard`

Phase 3: Matching & Skill Gap Analysis (`Compare Skills`, `Calculate Match Score`, `Identify Skill Gaps`)

12. **`Student`** → **`Frontend`** **(Solid Line):** `Select Job Listing to compare skills`
13. **`Frontend`** → **`Backend API`** **(Solid Line):** `POST /matching/evaluate/{student_id}/{job_id}`
14. **`Backend API`** → **`Database`** **(Solid Line):** `SELECT StudentSkills & JobRequirements`
15. **`Database`** ←-- **`Backend API`** **(Dashed Line):** `Return skill sets`
16. **Self-Message / Internal Execution Box in Backend:** `Calculate match_score % & identify missing skills (skill_gaps)`
17. **`Backend API`** → **`Database`** **(Solid Line):** `INSERT INTO "MatchResult" (match_score, eligibility_status)`
18. **`Database`** ←-- **`Backend API`** **(Dashed Line):** `MatchResult saved confirmation`
19. **`Backend API`** ←-- **`Frontend`** **(Dashed Line):** `Return JSON (match_score, skill_gaps, recommended_courses)`
20. **`Frontend`** ←-- **`Student`** **(Dashed Line):** `Display Match Badge & Skill Gap Breakdown`

Phase 4: AI Career Advice (`Student Requests AI Explanation`)

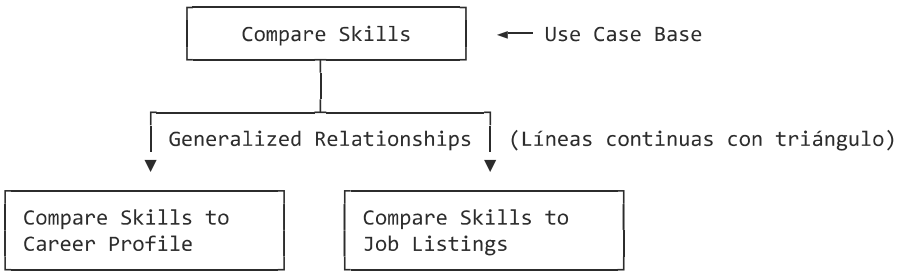
21. **`Student`** → **`Frontend`** **(Solid Line):** `Click "Ask AI to Explain Gap Results"`
22. **`Frontend`** → **`Backend API`** **(Solid Line):** `POST /ai/explain-gap`
23. **`Backend API`** → **`AI Explanation Service`** **(Solid Line):** `Send prompt payload with match_score & skill_gaps context`
24. **`AI Explanation Service`** ←-- **`Backend API`** **(Dashed Line):** `Return generated career advice text`
25. **`Backend API`** ←-- **`Frontend`** **(Dashed Line):** `200 OK (ai_insight_text)`

26. `Frontend` <-- `Student` **(Dashed Line):** `Render AI Career Insights Modal`

User prompt: sera que esto esta bien descrito

Response: **No del todo.** Aunque la idea básica tiene sentido, **la sintaxis de UML que estás usando en esa sección concreta tiene dos errores conceptuales importantes** que un evaluador o profesor de ingeniería de software te va a corregir inmediatamente.

1. Los 2 Errores Técnicos en la Imagen



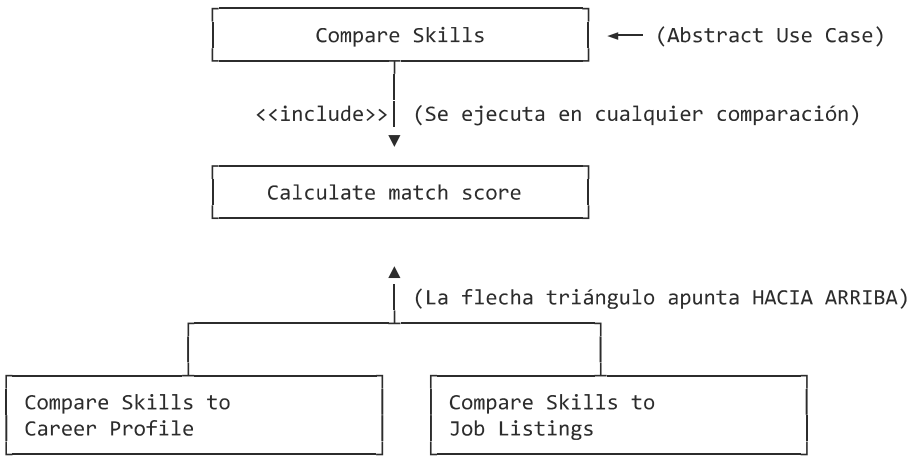
1. **Inversión de la Generalización / Herencia (Flecha sólida con triángulo ↑):**
   - En UML, la flecha de generalización apunta **de los casos de uso especializados hacia el caso de uso padre/general** (de abajo hacia arriba).
   - En tu gráfico, pusiste las flechas apuntando desde `Compare Skills` hacia abajo, lo que significaría que "Compare Skills es un tipo de Compare Skills to Job Listings", lo cual es **al revés.** Los casos de uso específicos (*Career Profile* y *Job Listings*) son especificaciones de `Compare Skills`.
2. **Conflicto entre Generalización e Includes (`<<include>>`):**
   - Si `Compare Skills` es un **caso de uso abstracto/padre** que se hereda a *Career Profile* y *Job Listings*, **no debería tener relaciones `<<include>>` directas** a `Calculate match score` e `Identify skill gaps`.
   - En UML, o bien tus dos subcasos específicos incluyen directamente esas tareas, o los subcasos heredan de `Compare Skills` y es `Compare Skills` el que ejecuta el include cuando es invocado.

2. Cómo Corregirlo Correctamente en UML

Para representarlo con rigor profesional en Lucidchart, tienes dos opciones estándar:

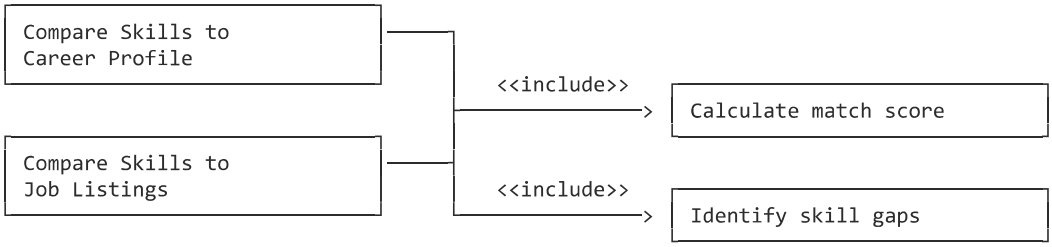
Opción A: Manteniendo la Herencia (Estructura Jerárquica Limpia)

Heredas la lógica base y apuntas los triángulos hacia el caso abstracto superior:



Opción B: Opción Más Sencilla y Directa (Recomendada)

En lugar de usar herencia/generalización (que suele complicar la lectura en entregables universitarios), conecta directamente los dos casos de uso concretos que realiza el usuario mediante `<<include>>` a sus componentes técnicos:



Resumen de Ajustes para Lucidchart

- **Dirección del triángulo en herencia:** Siempre apunta **hacia la clase/caso de uso padre** (△ arriba).
- **Relaciones `<<include>>`:** La flecha punteada siempre va **desde el caso de uso principal hacia el subcaso obligatorio** que necesita para funcionar (ej. de *Compare* ⇢ *Calculate*).

User prompt: entonces como cambiaria el secuecne diagram ?

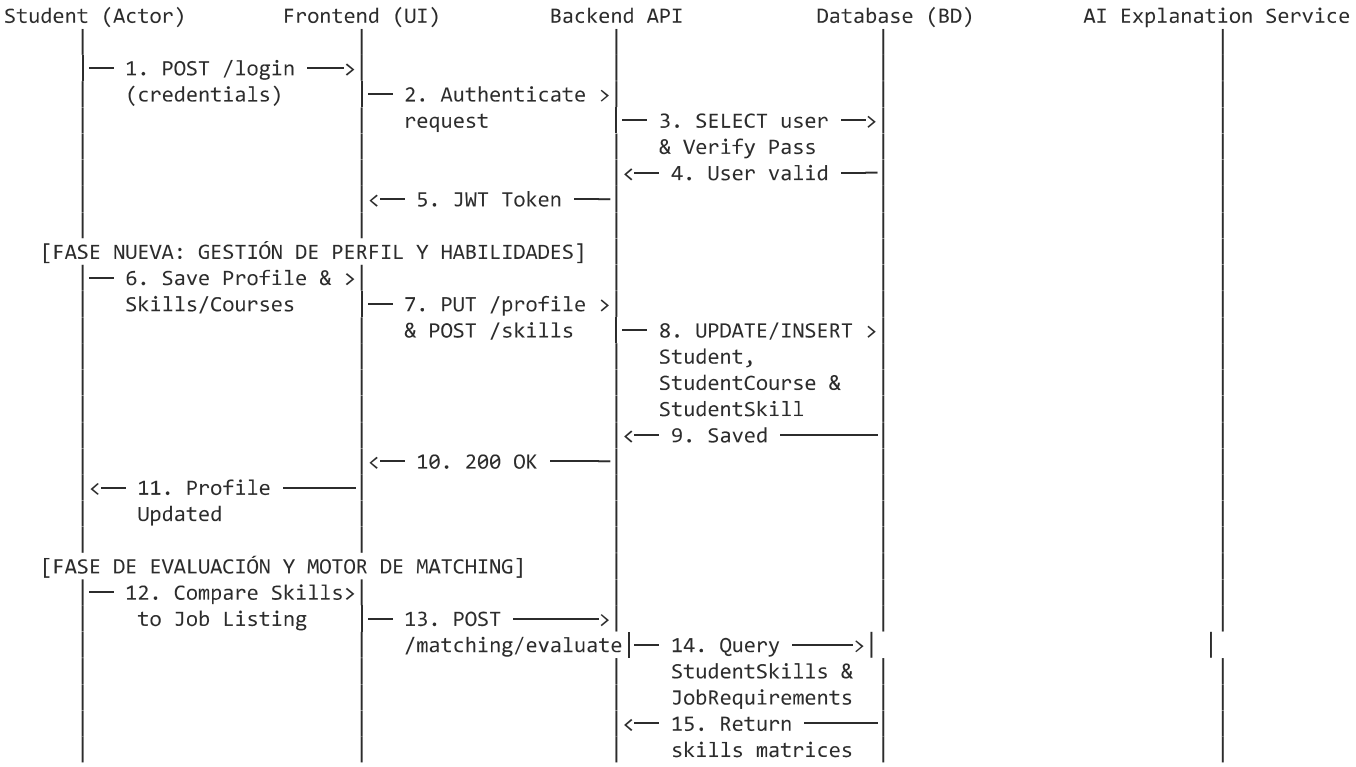
Response: Para que tu **Diagrama de Secuencia** (*Sequence Diagram*) refleje correctamente el ajuste que identificaste en tu mapa de Casos de Uso (donde el estudiante primero debe gestionar su perfil, sus asignaturas y sus *skills* antes de realizar comparaciones), la estructura del diagrama cambia incorporando ese paso intermedio de persistencia de datos.

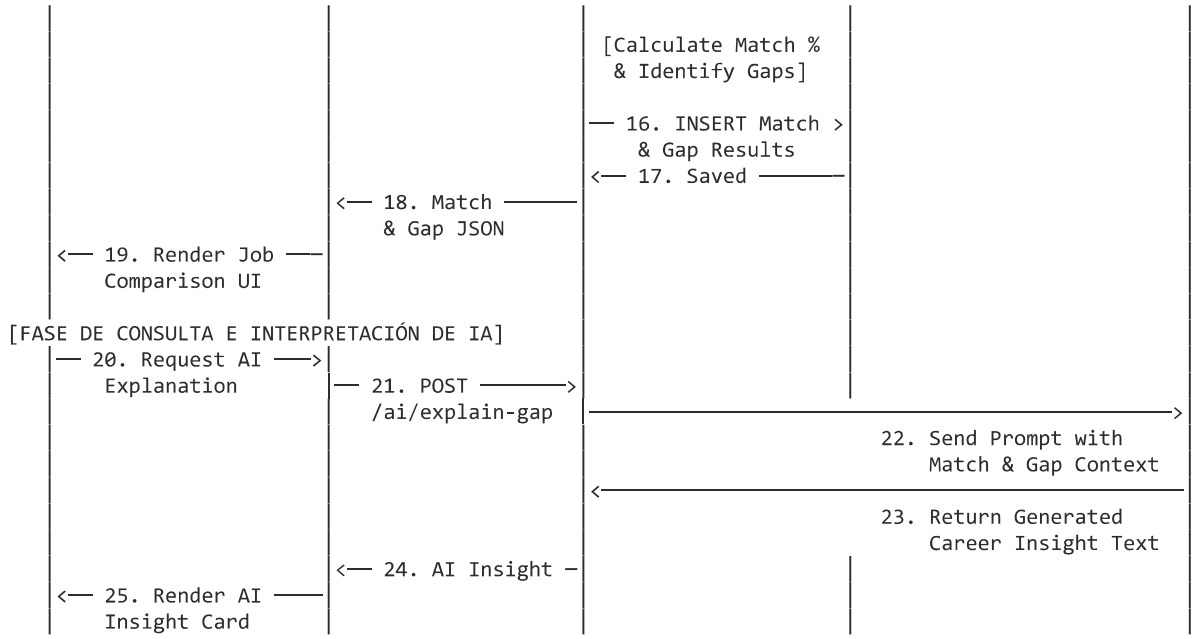
El cambio fundamental es que el flujo deja de ser directo de `Login` → `Compare Skills`, y pasa a ser un flujo completo de **4 fases ordenadas en el tiempo**:

¿Cómo cambia la secuencia? (Diagrama Final para Lucidchart)

En Lucidchart, mantendrás las mismas **5 Líneas de Vida (*Lifelines*)** organizadas en arquitectura por capas:

1. **`Student`** (Actor)
2. **`Frontend (UI)`** (Interfaz React/Web)
3. **`Backend API`** (FastAPI / Routers)
4. **`Database (BD)`** (PostgreSQL)
5. **`AI Service`** (Generador de resúmenes / insights)





Resumen del cambio paso a paso para construirlo en Lucidchart

1. **Agregas el bloque intermedio (Mensajes 6 al 11):**
   - Representa los casos de uso **`Manage Own Profile`** y **`Update Skills & Courses`**.
   - La llamada HTTP va como `PUT /profile` o `POST /skills` desde el Frontend hacia la API.
   - La API interactúa directamente con tus tablas de PostgreSQL: `STUDENT`, `StudentCourse` y `StudentSkill`.
2. **Garantizas la coherencia de datos para la Fase de Matching (Mensajes 12 al 19):**
   - Ahora, cuando la API ejecuta el endpoint `/matching/evaluate/{student_id}/{job_id}`, la consulta `SELECT` a la Base de Datos sí encuentra las habilidades que el usuario acaba de registrar/actualizar.
   - Se ejecuta la fórmula del algoritmo de compatibilidad y se guardan los resultados en la tabla `MatchResult` (`match_score` y `eligibility_status`).
3. **Invocación de Explicación de IA (Mensajes 20 al 25):**
   - Corresponde al caso de uso **`Student requests AI explanation`**.
   - El Backend toma los datos del *match* y las brechas calculadas (*skill gaps*) y se los envía como prompt al servicio de IA para devolver un texto interpretativo a la interfaz.

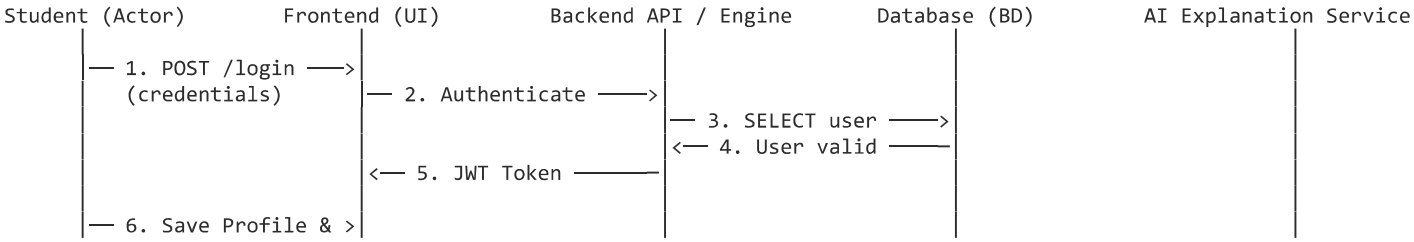
Con este orden cronológico, tu Diagrama de Secuencia guarda una **correspondencia exacta de 1 a 1** con los casos de uso que diseñaste en tu diagrama UML de casos de uso.

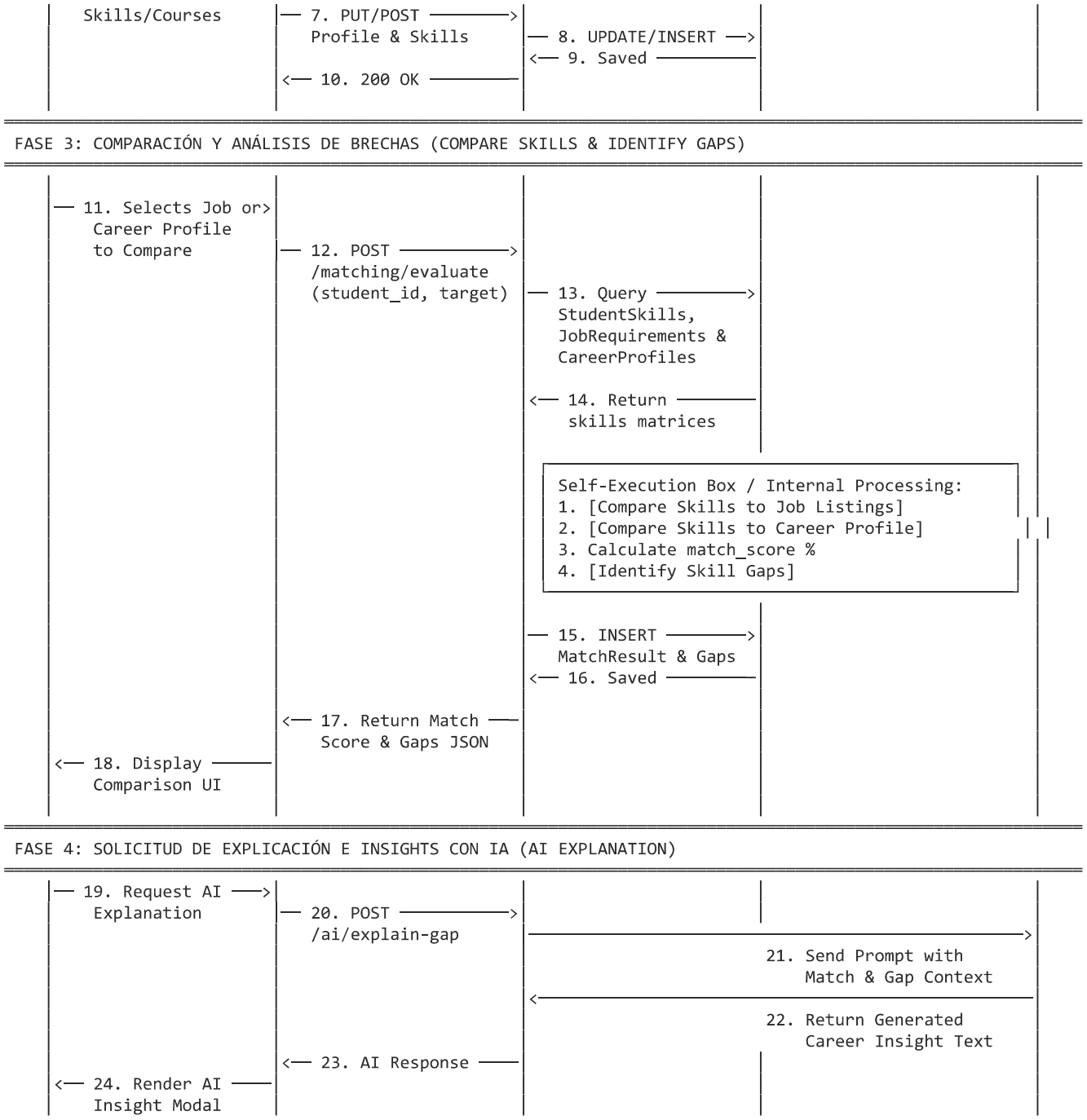
User prompt: DONDE QUEDQRIA COMPARE SKILLS TO CAREER PROFILE , COMPARE SKILLS TO JOB LISTINGS , Y IDENTIFY SKILLS GAPS ?

Response: En tu **Diagrama de Secuencia Completo** (Sequence Diagram), los tres casos de uso que mencionas se ubican estratégicamente dentro de las **Fases de Evaluación y Comparación**, ya que representan el núcleo algorítmico y de análisis de tu proyecto **Compass.**

A continuación te muestro **dónde y cómo se integran exactamente** en el flujo técnico del diagrama:

1. Ubicación exacta en la Secuencia





2. Explicación de los 3 componentes en el Diagrama

1. `Compare Skills to Job Listings` / `Compare Skills to Career Profile`:
   - **Ubicación:** Inicia en el **Mensaje 11 y 12** cuando el estudiante interactúa con la pantalla y el Frontend envía la petición `POST /matching/evaluate` a la API.
   - **En la Base de Datos (Mensaje 13 y 14):** La API consulta en paralelo las competencias guardadas del estudiante (`StudentSkill` + `StudentCourse`) y los requerimientos del objetivo elegido (`JobRequirement` para trabajos o `CareerProfile` para metas profesionales).
2. `Calculate match score` & `Identify Skill Gaps`:
   - **Ubicación:** Dentro del **caja de auto-ejecución/procesamiento interno (Self-Message / Internal Processing Box)** sobre la línea de vida del **`Backend API / Engine`**.
   - **Qué ocurre técnicamente:** El servidor compara los arreglos de habilidades. Resta las habilidades requeridas menos las habilidades del estudiante:
     - Las coincidencias generan la nota final (**`match_score %`**).

        - Las habilidades faltantes se extraen como la lista de brechas (`skill_gaps`).
    - **Persistencia (Mensajes 15 y 16):** Los resultados calculados se persisten en la base de datos en la tabla `MatchResult`.
3. Uso de la información de Gaps (Mensajes 17 a 24):
    - La API devuelve la información de la brecha al Frontend, permitiendo que la interfaz renderice el resultado gráfico y ofreciendo al estudiante la opción de invocar el servicio de IA (`Request AI Explanation`) para interpretar cómo cerrar esas brechas específicas (`skill_gaps`).
